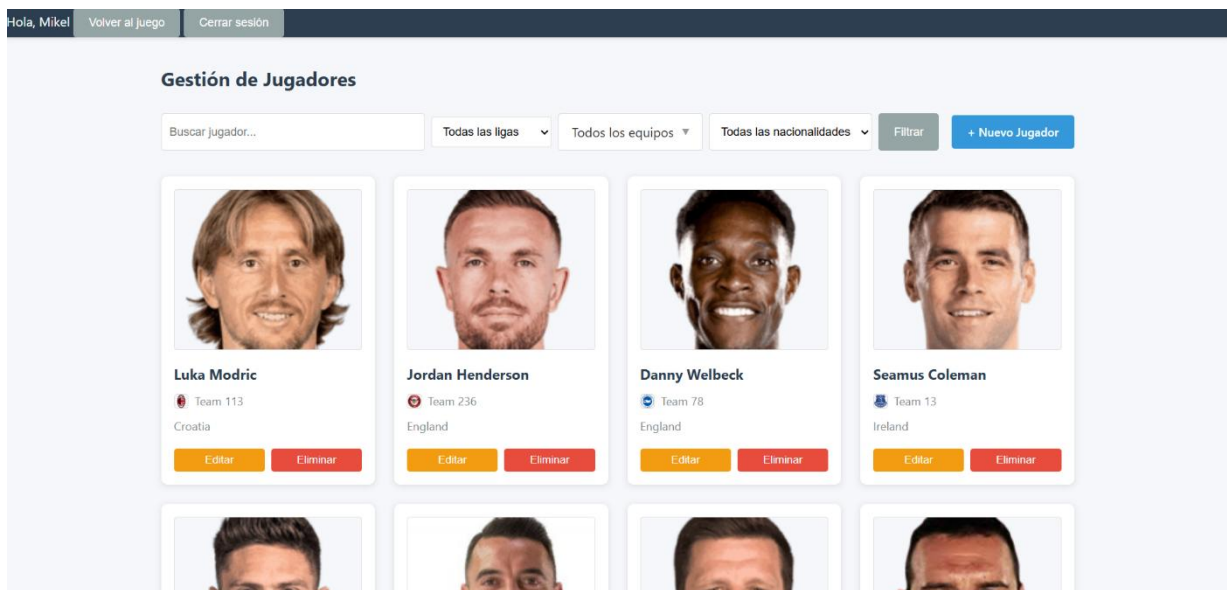


Backend:

Who are You?

Parte 2



Autores (GRUPO IX):

- Rubén Gallego
- Mikel Berasategui
- Oihane Pereira

Asignatura: Sistemas web

Curso académico: 2025-2026 Fecha: XX/01/2026

Github: https://github.com/RubenGGBC/WhoAreYaSW_Backend.git

Índice

Introducción	4
Milestone 0	5
1-Estructura del proyecto:	5
2-Punto de entrada del servidor:	5
3-Organización de carpetas:	6
4-Gestión de configuración:.....	6
5-Especificación: Rutas del sistema:.....	8
Milestone 1	14
1. Descargando logos de ligas (fetchLeagues.js).....	14
2. Descargando banderas (fetchNations.js).....	15
3. Descargando escudos de equipos (fetchTeams.js)	15
4. Descargando fotos de jugadores (fetchPlayers.js)	15
5. Bonus Track: El script unificado (fetchAll.js)	16
Milestone 2	17
1. Conexión a MongoDB (src/db/connection.js)	17
2. Esquema de League (src/models/League.js)	17
3. Esquema de Team (src/models/Team.js)	18
4. Esquema de Player (src/models/Player.js)	18
5. Script Seed (src/db/seeder/seedPlayers.js).....	18
6. Actualización de server.js.....	19
Milestone 3	20
1. Modelo User (src/models/User.js).....	20
2. Controlador de Autenticación (src/controllers/authController.js)	21
3. Rutas de Autenticación con Validación (src/routes/authRoutes.js)	22
4. Middlewares de Autenticación (src/middlewares/authMiddleware.js)	23
5. Sesiones con MongoStore (src/app.js)	23
6. Sistema de Roles.....	23
Milestone 4	25
1. Endpoints de la API.....	25
2. Endpoints del Juego	26
3. Validaciones con ifs en el Controlador	26

4. Integración con Frontend (loaders.js)	28
5. Testing con curl	28
6. Modelo Solution	29
7. Seeding de Soluciones.....	30
Milestone 5	31
1. Objetivos	31
2. Separación de rutas.....	31
3. Aproximación implementada (Cliente => API)	32
4. Dashboard (listado, búsqueda, filtros y paginación)	32
5. Formularios de creación/edición	32
6. Login/Register (EJS + fetch) y logout	33
7. Seguridad (sesiones, roles y contraseñas)	33
Milestone 6	35
1. OAuth con Google/GitHub (Passport.js)	35
2 Tests.....	38
3. JSON Web Tokens	41
WholsThisPokemon?	44
1. Localización y estructura:	44
2. Metodología de trabajo y documentación:.....	44
3. Cómo probarlo:	45
Uso de agentes	47
Conclusiones	48

Introducción

Esta parte del proyecto se ha realizado partiendo de la primera parte (frontend) como base de este:

- Enlace del informe técnico de la primera parte del proyecto (frontend): [WhoAreYa.docx](#).
- Enlace al repositorio github de la primera parte del proyecto (backend): <https://github.com/RubenGGBC/WhoAreYaSW>.

Esta segunda parte (backend) del proyecto se ha realizado siguiendo el orden, las indicaciones y los enunciados de cada milestone descritos en el fichero https://egela.ehu.eus/pluginfile.php/12014928/mod_resource/content/7/WhoAreYa_Back_End_.pdf. Se han realizado todos los ejercicios y milestones propuestos del proyecto. De hecho, incluso se ha realizado un apartado extra que no se solicitaba, el cual trata de adaptar la milestone 9 (<https://egela.ehu.eus/mod/page/view.php?id=9784872>) de la primera parte del proyecto (frontend) a esta segunda parte (backend), lo cual se describe en el apartado [WholsThisPokemon?](#).

Todos los cambios que se han ido realizando durante esta segunda parte (backend) del proyecto se han ido subiendo al repositorio de GitHub (https://github.com/RubenGGBC/WhoAreYaSW_Backend) y cada milestone ha quedado registrada en un tag del propio repositorio (https://github.com/RubenGGBC/WhoAreYaSW_Backend/tags), incluyendo el apartado extra realizado. Cabe destacar que el README.md del repositorio cambia en cada una de las tags, añadiendo en cada una de ellas la milestone realizada (tag de la milestone 0 -> README solo milestone 0, tag de la milestone 1 -> README milestone 0 y milestone 1, ...), sin embargo, en la entrega final el README queda tan solo con la milestone 0, tal y como se pide en el enunciado.

A continuación, en este documento, se explica cuáles han sido los cambios y avances importantes y destacables realizados o debatidos en cada una de las milestones y se incluye la dirección de los tags correspondientes a dichas milestones.

Milestone 0

- Tag de la milestone: https://github.com/RubenGGBC/WhoAreYaSW_Backend/releases/tag/backend-milestone0

1-Estructura del proyecto:

Opción elegida: **C - Estructura Híbrida**

Hemos optado por una estructura híbrida que combina lo mejor de ambos enfoques, tradicional y moderno.

Razones principales:

- Lo mejor de ambos mundos: Mantenemos public/ para servir archivos estáticos del frontend (convención estándar de Express), mientras organizamos el código del backend de forma modular en src/.
- Separación frontend/backend: El frontend existente permanece en public/, mientras que todo el código del servidor está organizado en src/.
- Escalabilidad: La estructura modular de src/ facilita el crecimiento del backend sin afectar al frontend.
- Convención de Express: Express busca archivos estáticos en public/ por defecto, lo que simplifica la configuración.
- Claridad: Es fácil de entender qué es frontend (public/) y qué es backend (src/), facilitando el trabajo en equipo.

2-Punto de entrada del servidor:

Opción elegida: **B - server.js**

Hemos optado por utilizar un solo archivo server.js para arrancar el servidor usando app.

Razones principales:

- Simplicidad y claridad: Un único archivo en la raíz hace que sea inmediatamente evidente dónde y cómo arranca la aplicación. Cualquier desarrollador que se una al proyecto puede identificar rápidamente el punto de inicio.
- Coherencia con estructura modular: Al usar estructura modular con `src/`, tiene sentido tener `server.js` en la raíz importando `src/app.js`.
- Separación de responsabilidades:
 - `server.js` se encarga exclusivamente de iniciar el servidor y manejar la configuración de inicio
 - `src/app.js` contiene la configuración completa de Express y los middlewares
 - Esta separación permite modificar la lógica de arranque sin afectar la configuración de la aplicación

3-Organización de carpetas:

Opción elegida: **A - Por tipo**

Hemos optado por agrupar el contenido del código por tipo.

Razones principales:

- Claridad: Es fácil localizar todos los modelos, todas las rutas o todos los controladores de un vistazo.
 - Todos los modelos en `/models`
 - Todas las rutas en `/routes`
 - Todos los controladores en `/controllers`
 - Toda la configuración en `/config`
- Mantenibilidad: Al tener pocas entidades principales (jugadores, partidas, estadísticas), agrupar por tipo resulta más eficiente que por dominio.
- Separación de responsabilidades: Cada carpeta tiene un propósito claro y único.

4-Gestión de configuración:

Opción elegida: **A – Usar .env directamente**

Razones principales:

- Proyecto pequeño: Al ser un proyecto pequeño con no demasiadas variables de entorno no es necesario preocuparse demasiado por la validación de estas. Tampoco hay demasiados archivos que requieran cargar las variables.
- Fácil de usar: La configuración escogida es cómoda y fácil de usar.
- Ejemplo en .env.example: Se define un .env.example en la raíz del proyecto, lo que guiará al usuario a crear su .env con rapidez, por lo que será fácil de usar para ellos también.

Variables de entorno por definir (.env y .env.example):

```
# Server Configuration
PORT
NODE_ENV
```

```
# Database
MONGO_URI
```

```
# Session
SESSION_SECRET
SESSION_MAX_AGE
```

```
# CORS
CORS_ORIGIN
```

```
# Game Configuration
SOLUTION_START_DATE
```

#A partir de aquí, para la milestone 6:

```
# OAuth Google
GOOGLE_CLIENT_ID
GOOGLE_CLIENT_SECRET
GOOGLE_CALLBACK_URL
```

```
# OAuth GitHub
GITHUB_CLIENT_ID
GITHUB_CLIENT_SECRET
GITHUB_CALLBACK_URL
```

```
# JWT Configuration
JWT_SECRET
JWT_EXPIRES_IN
JWT_REFRESH_SECRET
JWT_REFRESH_EXPIRES_IN
```

5-Especificación: Rutas del sistema:

- Rutas de API

Tip o	Métod o	Endpoint	Descripció n	Autorizació n
API	GET	/api/players	Obtener lista de todos los jugadores disponibles	Pública
API	GET	/api/players/:id	Obtener informació n detallada de un jugador específico	Pública
API	GET	/api/solution/:gameNumbe r	Obtener ID de la solución del juego (ofuscado)	Pública
API	POST	/api/stats	Guardar estadística s de una partida completad a	Pública
API	GET	/api/stats	Obtener estadística s globales del juego	Pública

API	GET	/api/game/current	Obtener número del juego actual basado en la fecha	Pública
API	GET	/api/game/:number	Obtener información de un juego específico por número	Pública

- Rutas de Vistas

Tipo	Método	Endpoint	Descripción	Autorización
Vista	GET	/	Página principal del juego del día actual	Pública
Vista	GET	/game/:number	Página del juego de un día específico	Pública

- Rutas de Administración (Futuras ampliaciones)

Tipo	Método	Endpoint	Descripción	Autorización
Admin	GET	/admin/players	Listar todos los jugadores con opciones de gestión	Admin
Admin	POST	/admin/players	Crear un nuevo jugador en la base de datos	Admin

Admin	PUT	/admin/players/:id	Actualizar información de un jugador existente	Admin
Admin	DELETE	/admin/players/:id	Eliminar un jugador de la base de datos	Admin
Admin	POST	/admin/solutions	Configurar las soluciones diarias del juego	Admin
Admin	GET	/admin/stats	Ver estadísticas detalladas del sistema	Admin

Estructura del Proyecto

```

WhoAreYaSW_Backend/
├─ `leagues.txt`
├─ `nationalities.txt`
├─ `package.json`
├─ `README.md`
├─ `server.js`
├─ `teamIDs.txt`
├─ `public/`
│   └─ `admin/`
│       ├── css/
│       │   └─ `admin.css`
│       └─ js/
│           ├── `admin-autocomplete.js`
│           ├── `admin-main.js`
│           ├── `api-client.js`
│           ├── `custom-select.js`
│           ├── `player-edit.js`
│           └─ `player-form.js`
├─ `auth/`
│   ├── css/
│   │   ├── `auth.css`
│   │   └─ `oauth.css`
│   └─ js/
│       ├── `login.js`
│       └─ `register.js`
├─ images/
└─ leagues/

```

```

├── | └─ ...
├── | └─ nations/
├── | | └─ ...
├── | └─ players/
├── | └─ teams/
├── └─ frontend (parte 1 de de WhoAreYa)
├── └─ src/
├── | └─ app.js`
├── | └─ config/
├── | | └─ `multer.js`
├── | | └─ `passport.js`
├── | └─ controllers/
├── | | └─ `adminController.js`
├── | | └─ `authController.js`
├── | | └─ `gameController.js`
├── | | └─ `oauthController.js`
├── | | └─ `oauthRoutes.js`
├── | └─ db/
├── | | └─ `connection.js`
├── | | └─ seeders/
├── | | | └─ `seedPlayers.js`
├── | | | └─ `seedSolutions.js`
├── | └─ middlewares/
├── | | └─ `authMiddleware.js`
├── | └─ models/
├── | | └─ `League.js`
├── | | └─ `Player.js`
├── | | └─ `Solution.js`
├── | | └─ `Stats.js`
├── | | └─ `Team.js`
├── | | └─ `User.js`
├── | └─ routes/
├── | | └─ `adminRoutes.js`
├── | | └─ `authRoutes.js`
├── | | └─ `gameRoutes.js`
├── | | └─ `oauthRoutes.js`
├── | | └─ `playerRoutes.js`
├── | └─ scripts/
├── | | └─ `fetchAll.js`
├── | | └─ `fetchLeagues.js`
├── | | └─ `fetchNations.js`
├── | | └─ `fetchPlayers.js`
├── | | └─ `fetchTeams.js`
├── | └─ utils/
├── | | └─ `jwt.js`
├── └─ tests/
├── | └─ `auth-routes.test.js`
├── | └─ `auth.test.js`
├── | └─ `controllers.test.js`
├── | └─ `debug-auth.test.js`
├── | └─ `find-auth-routes.test.js`
├── | └─ `player-model.test.js`
├── | └─ `setup.js`
├── | └─ `test-app.js`
├── └─ views/
├── | └─ admin/
├── | | └─ `dashboard.ejs`

```

```

| |   ├── `edit-player.ejs`
| |   └── `new-player.ejs` |
|   ├── auth/
|   |   ├── `login.ejs`
|   |   └── `register.ejs`
|   └── partials/
|       ├── `admin-footer.ejs`
|       └── `admin-header.ejs`
└── WholsThisPokemon/
    ├── Juego de pokemon (misma estructura que WhoAreYa)

```

Dependencias del Proyecto (package.json)

```

{
  "name": "WhoAreYa",
  "version": "1.0.0",
  "description": "",
  "main": "public/index.js",
  "scripts": {
    "test": "cross-env NODE_ENV=test jest --verbose",
    "test:watch": "cross-env NODE_ENV=test jest --watch"
  },
  "private": true,
  "dependencies": {
    "autosuggest-highlight": "^3.3.4",
    "bcryptjs": "^3.0.3",
    "browserify": "^17.0.1",
    "connect-mongo": "^6.0.0",
    "dotenv": "^17.2.3",
    "ejs": "^3.1.10",
    "express": "^5.2.1",
    "express-session": "^1.18.2",
    "express-validator": "^7.3.1",
    "jsonwebtoken": "^9.0.3",
    "mongoose": "^9.0.2",
    "multer": "^2.0.2",
    "node-fetch": "^2.6.1",
    "passport": "^0.7.0",
    "passport-github2": "^0.1.12",
    "passport-google-oauth20": "^2.0.0"
  },
  "devDependencies": {
    "cross-env": "^10.1.0",
    "jest": "^30.2.0",
    "mongodb-memory-server": "^11.0.1",
    "supertest": "^7.1.4"
  }
}

```

- **Instalación y Configuración**

- **Requisitos previos:**

Node.js

MongoDB

npm

- **Pasos de instalación:**

1. Clonar el repositorio:

```
git clone https://github.com/tu-usuario/WhoAreYa-backend.git  
cd whoareya-backend
```

2. Instalar dependencias:

```
npm install
```

3. Configurar variables de entorno:

```
cp .env.example .env  
# Editar .env con tus valores
```

4. Inicializar la base de datos:

```
npm run seed
```

5. Iniciar el servidor:

```
node server.js
```

El servidor estará disponible en <http://localhost:3000> (o en el que se haya configurado en las variables de entorno correspondientes).

****NOTA:**** Inicialmente, la Milestone 0 era diferente (era algo provisional), en este documento se ha ido cambiando la información necesaria a medida que avanzaban las milestones y las necesidades cambiaban. Si se quiere ver como era la milestone 0 inicialmente, ver el README de la tag “backend-milestone0” del repositorio.

Milestone 1

- Tag de la milestone: https://github.com/RubenGGBC/WhoAreYaSW_Backend/releases/tag/backend-milestone1

¿Qué hemos hecho aquí?

Para empezar, hemos creado scripts que nos permiten descargar todos los datos que usamos en la aplicación. De este modo, en vez de estar llamando siempre a APIs externas (que pueden caerse o ir lentas), descargamos todo a nuestro servidor y lo servimos desde ahí. Así la app va más rápida y no dependemos de nadie.

Los scripts que hemos creado:

Hemos creado varios scripts en `src/scripts/` para descargar todo:

`src/scripts/`

- |— `fetchLeagues.js` → Descarga logos de las 5 ligas
- |— `fetchNations.js` → Descarga banderas de países
- |— `fetchTeams.js` → Descarga escudos de equipos
- |— `fetchPlayers.js` → Descarga fotos de jugadores
- |— `fetchAll.js` → Script todo-en-uno (bonus)

Todo se guarda en `public/images/` organizado por tipo (leagues, nations, teams, players).

1. Descargando logos de ligas (`fetchLeagues.js`)

Lee un archivo `leagues.txt` con 5 líneas (`de1`, `en1`, `es1`, `fr1`, `it1`) y descarga el logo de cada liga.

Puntos destacables:

¿Por qué usamos `pipe()` en el código? Porque las imágenes pueden ser grandes. Con `pipe()` los datos van directamente del servidor al disco, byte a byte, sin tener que cargar toda la imagen en la RAM primero.

Ejecutar: `node src/scripts/fetchLeagues.js`

2. Descargando banderas (fetchNations.js)

Este es casi igual que el anterior, pero tiene un detalle importante: algunos países tienen espacios en el nombre.

Puntos destacables:

¿Qué hace encodeURIComponent en el código? Convierte caracteres especiales (espacios, acentos, etc.) en códigos que se pueden usar en URLs. Los espacios se convierten en %20, por ejemplo.

Resultado: 102 de 103 banderas descargadas (había una línea vacía que ignoramos)

3. Descargando escudos de equipos (fetchTeams.js)

Esta es más diferente al resto: la URL para descargar un escudo no es simplemente el ID del equipo. Hay que hacer un cálculo matemático primero

Puntos destacables:

¿Por qué usamos módulo 32 en el código? Porque la API organiza las imágenes en 32 carpetas (0 a 31) para no tener miles de archivos en un solo directorio. El operador % (módulo) nos da el resto de dividir entre 32. Así:

$33 \% 32 = 1 \rightarrow$ va a la carpeta 1

$64 \% 32 = 0 \rightarrow$ va a la carpeta 0

$50 \% 32 = 18 \rightarrow$ va a la carpeta 18

Con esto podemos descargar todos los escudos correctamente.

4. Descargando fotos de jugadores (fetchPlayers.js)

Aquí tenemos 2038 jugadores. Si intentas descargar todo de golpe, el servidor te bloquea con un error 429 (Too Many Requests).

La solución: Throttling

Limitamos las peticiones a 10 por segundo:

```
const REQUESTS_PER_SECOND = 10;
const DELAY_MS = 1000 / REQUESTS_PER_SECOND; // 100ms entre cada petición
```

Puntos destacables:

¿Cómo funciona el **throttling**?

- Usamos `setTimeout()` para retrasar cada petición
- La primera petición sale inmediatamente ($0 * 100\text{ms} = 0\text{ms}$).
- La segunda espera 100ms
- La tercera espera 200ms
- Y así...

Resultado: enviamos 10 peticiones por segundo exacto

Tiempo estimado: Unos 3-4 minutos para los 2038 jugadores.

5. Bonus Track: El script unificado (fetchAll.js)

Como el código de los otros scripts es repetitivo y puede ser pesado ir ejecutándolos de uno en uno, hemos decidido juntar todos ellos en 1.

Ejecutar: `node src/scripts/fetchAll.js leagues` (o `nations`, `teams`, `players`)

Archivos que necesitas tener

Para que los scripts funcionen, necesitas estos archivos en la raíz del proyecto:

- `leagues.txt` - Bajar de [aquí](#)
- `nationalities.txt` . Generarlo con: `jq -er 'map(.nationality) | .[]' fullplayers25.json | Sort-Unique > nationalities.txt`
- `teamIDs.txt` - Generarlo con: `jq -r 'map(.teamId) | unique | sort | .[]' fullplayers25.json > teamIDs.txt`
- `public/json/fullplayers25.json` - Esta en el frontend original.

Milestone 2

- Tag de la milestone: https://github.com/RubenGGBC/WhoAreYaSW_Backend/releases/tag/backend-milestone2

¿Qué hemos hecho?

Conectamos la app a MongoDB y creamos los esquemas (moldes) para nuestros datos. Así, MongoDB sabe exactamente cómo deben verse los jugadores, equipos y ligas.

Archivos creados

src/

└─ db/

| └─ connection.js ← Conexión a MongoDB

| └─ seeders/

| └─ seedPlayers.js ← Script para llenar la BD

|

└─ models/

└─ League.js ← Esquema de ligas

└─ Team.js ← Esquema de equipos

└─ Player.js ← Esquema de jugadores

1. Conexión a MongoDB (src/db/connection.js)

Este archivo conecta tu app con MongoDB usando Mongoose.

¿Qué hace?

- mongoose.connect() → Se conecta a MongoDB
- Si falla → process.exit(1) mata el servidor (no queremos un app sin BD)
- Si funciona → Console dice que está conectado

2. Esquema de League (src/models/League.js)

Define cómo se ve una liga en la BD.

Validaciones:

- id → Número único (no puede haber dos con el mismo)
- name → String obligatorio, mínimo 2 caracteres
- code → Código único (ej: 'de1', 'en1')
- country y flagUrl → Opcionales

3. Esquema de Team (src/models/Team.js)

Define cómo se ve un **equipo** en la BD.

Validaciones:

- id → Número único
- name → String obligatorio, mínimo 2 caracteres
- leagueId → Número obligatorio (relaciona el equipo con una liga)
- Resto → Opcionales

4. Esquema de Player (src/models/Player.js)

Define cómo se ve un **jugador** en la BD.

Validaciones

- position → Usa enum para limitar a solo 4 valores: DF (defensa), MF (mediocampo), FW (delantero), GK (portero)
- Si intentas guardar position: "INVALID" → MongoDB rechaza
- Solo acepta los 4 valores válidos

5. Script Seed (src/db/seeder/seedPlayers.js)

Este script llena la BD con los 2038 jugadores que descargamos en el Milestone 1.

¿Qué hace?

1. Conecta a MongoDB
2. Borra datos viejos (si existen)
3. Lee el archivo fullplayers25.json
4. Extrae ligas y equipos únicos
5. Inserta todo en la BD
 - 4 ligas
 - 78 equipos
 - 2038 jugadores

Ejecutar:

```
node src/db/seeder/seedPlayers.js
```

Output esperado:

Conectado a MongoDB: mongodb://localhost:27017/whoareya

Insertando 4 ligas...

Insertando 78 equipos...

Insertando 2038 jugadores...

Base de datos poblada correctamente

- 4 ligas
- 78 equipos
- 2038 jugadores

6. Actualización de server.js

Para que todo funcione, server.js debe llamar a connectDB():

Milestone 3

- Tag de la milestone: https://github.com/RubenGGBC/WhoAreYaSW_Backend/releases/tag/backend-milestone3

¿Qué hemos hecho?

En este milestone implementamos un sistema completo de autenticación y autorización. Permite que los usuarios se registren, inicien sesión, y accedan a recursos protegidos según su rol.

Cambios importantes:

- Los usuarios se pueden registrar y hacer login/logout
- Las contraseñas se guardan hasheadas (seguridad)
- Las sesiones se almacenan en MongoDB (persisten entre reinicios)
- Hay dos roles: admin y user
- El **primer usuario que se registra automáticamente es admin**
- Tenemos middlewares para proteger rutas (solo usuarios autenticados, solo admins, etc.)

1. Modelo User (src/models/User.js)

Define cómo se ve un **usuario** en la BD.

Validaciones:

- name y lastName → Strings obligatorios, mínimo 2 caracteres
- email → Único, validado con regex para que sea un email válido
- password → Mínimo 8 caracteres, se hashea automáticamente antes de guardar
- role → Solo puede ser 'admin' o 'user'

Cuando el usuario registra su contraseña "password123", no la guardamos tal cual. En su lugar:

1. Se genera un "salt" (número aleatorio) - línea `const salt = await bcrypt.genSalt(10)`
2. Se hashea la contraseña combinada con el salt - línea `await bcrypt.hash(this.password, salt)`
3. Se guarda el hash en la BD (la contraseña original se pierde)

Cuando el usuario intenta login, usamos `comparePassword()` para comparar la contraseña que envía con el hash guardado. Si coinciden, el login es válido.

De esta manera podemos ocultar las contraseñas de los usuarios en la BD y solo se ve la contraseña hasheada

2. Controlador de Autenticación (src/controllers/authController.js)

El controlador maneja 4 operaciones:

1- register(req, res)

Crea un nuevo usuario. Verifica que el email no exista y que sea el primero si es admin.

Validaciones:

1. El email no debe existir
2. Las dos contraseñas deben coincidir
3. Se valida nombre, apellido, email, contraseña (en las rutas, ver paso 3)

2- login(req, res)

Verifica email y contraseña, crea una sesión.

¿Qué pasa internamente?

1. Se busca el usuario por email
2. Se compara la contraseña con comparePassword()
3. Si es correcta, se crea una sesión en MongoDB
4. El navegador recibe una cookie connect.sid con el ID de la sesión

3- logout(req, res)

Destruye la sesión en MongoDB y borra la cookie.

Requiere estar autenticado (tener sesión activa).

4- getCurrentUser(req, res)

Devuelve quién eres si estás logueado.

Requiere estar autenticado.

3. Rutas de Autenticación con Validación (src/routes/authRoutes.js)

Las rutas usan express-validator para validar los datos antes de llegar al controlador.

Rutas públicas:

POST /auth/register // Crear cuenta

POST /auth/login // Iniciar sesión

Rutas protegidas:

POST /auth/logout // Requiere sesión activa

GET /auth/me // Requiere sesión activa

Validaciones en /register:

- name: Mínimo 2 caracteres
- lastName: Mínimo 2 caracteres
- email: Debe ser un email válido (usando .isEmail())
- password: Mínimo 8 caracteres
- confirmPassword: Debe coincidir exactamente con password

Validaciones en /login:

- email: Debe ser email válido
- password: Es obligatorio

Si hay errores de validación, se retorna **400**

4. Middlewares de Autenticación (src/middlewares/authMiddleware.js)

Dos middlewares que protegen las rutas:

1- **isAuthenticated**

Verifica que exista sesión activa.

2- **isAdmin**

Verifica que el rol sea 'admin'. Se usa junto con isAuthenticated.

5. Sesiones con MongoStore (src/app.js)

Las sesiones se configuran con MongoStore para que persistan en MongoDB:

Ventajas de MongoStore:

- Las sesiones se guardan en MongoDB
- Persisten si reinicia el servidor
- Funcionan con múltiples servidores
- El usuario sigue logueado tras un restart
- Sin MongoStore, las sesiones se pierden al reiniciar

Variables de entorno necesarias:

MONGO_URI

SESSION_SECRET

6. Sistema de Roles

Dos roles:

- admin → Acceso a CRUD de jugadores, panel de administración, etc.
- user → Solo lectura de datos públicos

Regla especial: El primer usuario que se registra automáticamente es admin. Los siguientes son user.

Milestone 4

- Tag de la milestone: https://github.com/RubenGGBC/WhoAreYaSW_Backend/releases/tag/backend-milestone4

¿Qué hemos hecho?

En este milestone implementamos una API REST completa para gestionar jugadores. El backend ahora sirve datos desde la BD en lugar de archivos JSON estáticos.

Cambios importantes:

- 7 endpoints REST para operaciones CRUD (getPlayers, getPlayersById, createPlayer, updatePlayer, deletePlayer, getTeams, getLeagues)
- Rutas públicas para lectura (GET) sin autenticación requerida
- Rutas protegidas para escritura (POST, PUT, DELETE) - solo rol admin
- Paginación en listados de jugadores
- Validaciones manuales con ifs en los métodos del controlador
- Manejo de errores consistente con códigos de error específicos
- Endpoints específicos para el juego (número actual, información, solución del día)
- Separación clara entre controladores (lógica) y rutas (HTTP)

1. Endpoints de la API

Rutas públicas (sin autenticación):

- **GET /api/players** Obtiene lista paginada de jugadores.
- **GET /api/players/:id** Obtiene un jugador específico por su ID de MongoDB.
- **GET /api/teams** Obtiene lista de todos los equipos.
- **GET /api/leagues** Obtiene lista de todas las ligas.
- **GET /api/game/current** Obtiene el número del juego actual basado en la fecha actual.

- **GET /api/game/:gameNumber** Obtiene información de un juego específico por número.
- **GET /api/solution/:gameNumber** Obtiene la solución del día (el jugador del día) para un juego específico.

Rutas protegidas (requieren admin):

- **POST /api/players** Crea un nuevo jugador. Solo administrador.
- **PUT /api/players/:id** Actualiza un jugador existente. Solo administrador.
- **DELETE /api/players/:id** Elimina un jugador. Solo administrador.

2. Endpoints del Juego

- **GET /api/game/current - Número de juego actual:**

Calcula automáticamente basándose en la fecha actual y SOLUTION_START_DATE.

La fórmula es:

```
const diffDays = Math.floor((now - SOLUTION_START_DATE) / (1000 * 60 * 60 * 24));
```

```
const gameNumber = diffDays + 1;
```

- **GET /api/game/:gameNumber - Información del juego**
- **GET /api/solution/:gameNumber - Solución del día**

Obtiene el jugador que es la solución para un día específico.

3. Validaciones con ifs en el Controlador

Las validaciones se implementan directamente en los métodos del controlador usando ifs, en lugar de middleware externo.

Ejemplo: validar id en createPlayer() y updatePlayer():

```
const errors = [];  
  
// Validar id  
if (!id) {  
  errors.push('El ID del jugador es requerido');  
} else if (typeof id !== 'number' && isNaN(id)) {  
  errors.push('El ID debe ser un número');  
}
```

Validaciones en cada operación

-En createPlayer:

- ID es requerido y debe ser numérico
- Name es requerido y mínimo 2 caracteres
- Position es requerida y debe ser DF/MF/FW/GK
- ID debe ser único en la BD
- Requiere autenticación + rol admin

-En updatePlayer:

- Mismas validaciones que createPlayer
- Si el jugador no existe → 404
- Requiere autenticación + rol admin

-En deletePlayer:

- Si el jugador no existe → 404
- Requiere autenticación + rol admin

-En GET (/players, /players/:id, /teams, /leagues):

- Sin validaciones requeridas
- Sin autenticación requerida
- Acceso público

4. Integración con Frontend (loaders.js)

El archivo `public/js/loaders.js` ha sido actualizado para consumir la API

Flujo de uso:

1. Frontend llama `fetchJSON('fullplayers25')` → Backend devuelve array de jugadores desde BD
2. Frontend llama `fetchJSON('solution25')` → Obtiene `gameNumber` actual → Obtiene solución del día
3. Frontend llama `fetchPlayer(playerId)` → Obtiene detalles de un jugador para comparar

5. Testing con curl

Listar jugadores (página 1, 10 items)

```
curl http://localhost:3000/api/players?page=1&limit=10
```

Obtener un jugador por ID

```
curl http://localhost:3000/api/players/507f1f77bcf86cd799439011
```

Listar equipos

```
curl http://localhost:3000/api/teams
```

Listar ligas

```
curl http://localhost:3000/api/leagues
```

Obtener número de juego actual

```
curl http://localhost:3000/api/game/current
```

Obtener información del juego 1

```
curl http://localhost:3000/api/game/1
```

Obtener solución del día 1

```
curl http://localhost:3000/api/solution/1
```

Crear jugador (requiere estar logeado como admin)

```
curl -X POST http://localhost:3000/api/players \
```

```
-H "Content-Type: application/json" \
```

```
-b cookies.txt \
```

```
-d '{
```

```
  "id": 99999,
```

```
  "name": "Test Player",
```

```
  "position": "FW",
```

```
  "birthDate": "1990-01-01",
```

```
  "nationality": "Spain",
```

```
  "teamId": 2030,
```

```
  "leagueId": 8
```

```
}'
```

Actualizar jugador

```
curl -X PUT http://localhost:3000/api/players/507f1f77bcf86cd799439011 \
```

```
-H "Content-Type: application/json" \
```

```
-b cookies.txt \
```

```
-d '{
```

```
  "name": "Updated Name",
```

```
  "number": 10
```

```
}'
```

Eliminar jugador

```
curl -X DELETE http://localhost:3000/api/players/507f1f77bcf86cd799439011 \
```

```
-b cookies.txt
```

6. Modelo Solution

Define cómo se ve una **solución** en la BD (el jugador del día).

Campos:

- gameNumber → Número único del juego (1, 2, 3, ...)

- playerId → ID del jugador que es la solución
- date → Fecha correspondiente al juego

7. Seeding de Soluciones

Se creó el script **seedSolutions.js** que:

1. Crea una solución para cada jugador en la BD
2. Asigna un número de juego (gameNumber)
3. Calcula automáticamente la fecha basada en SOLUTION_START_DATE
4. Los jugadores se asignan secuencialmente (día 1 = jugador 1, día 2 = jugador 2, etc.)

Ejecutar:

```
node src/db/seeder/seedPlayers.js # Primero los jugadores
```

```
node src/db/seeder/seedSolutions.js # Luego las soluciones
```

Milestone 5

- Tag de la milestone: https://github.com/RubenGGBC/WhoAreYaSW_Backend/releases/tag/backend-milestone5

¿Qué hemos hecho?

Este milestone implementa un panel web para administradores que permite gestionar (CRUD) los jugadores de forma visual, consumiendo la API REST del Milestone 4. La implementación mantiene una separación clara entre juego (/), API (/api/) y el panel (/admin/).

1. Objetivos

- **Vistas del panel con EJS**
 - views/admin/dashboard.ejs
 - views/admin/new-player.ejs
 - views/admin/edit-player.ejs
 - Parciales: views/partials/admin-header.ejs, views/partials/admin-footer.ejs
- **Formularios HTML para CRUD**
 - Alta/edición con formularios HTML.
 - Borrado desde el listado con confirmación.
- **Frontend conectado a la API REST con fetch**
 - Cliente común: public/admin/js/api-client.js.
 - Operaciones consumidas:
 - GET /api/players (con listado con paginación + filtros añadidos)
 - GET /api/players/:id
 - POST /api/players
 - PUT /api/players/:id
 - DELETE /api/players/:id
 - GET /api/teams, GET /api/leagues
- **Protección de rutas (solo admin)**
 - /admin/* protegido por sesión + rol admin mediante middleware.
 - /api/players (POST/PUT/DELETE) también requiere admin.
- **Feedback visual**
 - Mensajes de éxito/error en dashboard y formularios.
 - Confirmación antes de eliminar.

2. Separación de rutas

- /* → juego + vistas públicas y autenticación.

- `/api/*` → API REST (JSON). Lectura pública, escritura protegida.
- `/admin/*` → panel de administración (requiere autenticación + rol admin).

3. Aproximación implementada (Cliente => API)

Se ha implementado la **Aproximación 2** del enunciado:

- El servidor solo renderiza vistas **GET** del panel.
- El navegador ejecuta el CRUD llamando directamente a `/api/*` mediante fetch.

Rutas del panel:

- **GET /admin** → Dashboard.
- **GET /admin/players/new** → Formulario de creación.
- **GET /admin/players/edit/:id** → Formulario de edición.

4. Dashboard (listado, búsqueda, filtros y paginación)

Implementado en **views/admin/dashboard.ejs** + **public/admin/js/admin-main.js**:

- Búsqueda (search).
- Filtros por liga (league) y nacionalidad (nationality).
- Paginación.
- Acciones:
 - Editar → navega a `/admin/players/edit/:id`.
 - Eliminar → confirmación + DELETE `/api/players/:id`.

5. Formularios de creación/edición

- **Crear jugador: /admin/players/new**
 - JS: `public/admin/js/player-form.js`.
 - Carga equipos/ligas desde API para rellenar selects.
 - Validación en cliente (HTML5 + JS): campos obligatorios y fecha razonable.

- **Editar jugador: /admin/players/edit/:id**
 - JS: public/admin/js/player-edit.js.
 - Carga el jugador con GET /api/players/:id.
 - Mantiene el id numérico del jugador al guardar.

6. Login/Register (EJS + fetch) y logout

Para soportar el panel, se añadieron vistas de autenticación:

- **GET /login** → views/auth/login.ejs
- **GET /register** → views/auth/register.ejs

Y su lógica de cliente:

- public/auth/js/login.js → **POST /login**.
- public/auth/js/register.js → **POST /register**.

Tras login/registro correcto:

- **Se crea sesión** (cookie connect.sid).
- El cliente **redirige**:
 - si role === 'admin' → /admin
 - si no → /

Logout:

- Desde el panel: cabecera views/partials/admin-header.ejs.
- Botón “Cerrar sesión” llama a POST /logout.
- Para mostrar el usuario actual se usa GET /current-user.

7. Seguridad (sesiones, roles y contraseñas)

- **Sesiones**: express-session + persistencia en MongoDB con connect-mongo.
- **Roles**: admin y user.

- Regla del proyecto: el **primer usuario registrado** se crea con rol admin; el resto con rol user.
- **Contraseñas hasheadas:**
 - En src/models/User.js se usa bcryptjs con un hook pre('save') para hashear la contraseña.
 - En login se verifica con comparePassword() (bcrypt compare).

****Notas**:**

1. El panel solo es accesible para usuarios con rol admin.
2. El admin será el primer usuario registrado en la aplicación. El resto serán user.
3. Para probarlo, seguir estos pasos:
 - Definir las variables de entorno en .env.
 - Instalar dependencias.
 - Iniciar MongoDB localmente.
 - Correr los seeds de jugadores y soluciones (src/db/seeder/seedPlayers.js y src/db/seeder/seedSolutions.js).
 - Iniciar el servidor.
 - Acceder a /register.
 - Registrar el primer usuario → será admin.
 - Acceder al panel /admin.
 - Ver el dashboard, crear/editar/borrar jugadores.
 - Cerrar sesión.
 - Registrar más usuarios → serán user y no podrán acceder al panel.

Milestone 6

- Tag de la milestone: https://github.com/RubenGGBC/WhoAreYaSW_Backend/releases/tag/backend-milestone6

1. OAuth con Google/GitHub (Passport.js)

¿Qué hemos hecho?

Hemos implementado un sistema completo de autenticación OAuth con Google y GitHub, integrando Passport.js con nuestro sistema de autenticación existente basado en sesiones.

1. Configuración de Passport.js

- Instalación de dependencias: passport, passport-google-oauth20, passport-github2
- Creación del archivo src/config/passport.js con estrategias para Google y GitHub+

2. Configuración de estrategias OAuth

- Google OAuth: Configuración para obtener perfil y email del usuario
- GitHub OAuth: Manejo especial para cuentas con email privado
- Lógica para crear automáticamente usuarios nuevos cuando se autentican por primera vez

3. Rutas de autenticación

Creación de src/routes/oauthRoutes.js con endpoints:

- GET /auth/google - Inicia flujo de Google OAuth
- GET /auth/google/callback - Callback de Google
- GET /auth/github - Inicia flujo de GitHub OAuth
- GET /auth/github/callback - Callback de GitHub

4. Integración con el sistema existente

- Los usuarios OAuth se integran automáticamente con el modelo User existente

- Asignación automática de rol 'user' a nuevos usuarios OAuth
- Redirección inteligente: admin → panel, user → home

5. Interfaz de usuario

- Botones de "Login with Google" y "Login with GitHub" en la página de login
- Estilos CSS personalizados para los botones OAuth
- Experiencia de usuario fluida con redirección automática

Configuración necesaria:

Google OAuth

GOOGLE_CLIENT_ID=tu_client_id_google

GOOGLE_CLIENT_SECRET=tu_client_secret_google

GOOGLE_CALLBACK_URL=http://localhost:3000/auth/google/callback

GitHub OAuth

GITHUB_CLIENT_ID=tu_client_id_github

GITHUB_CLIENT_SECRET=tu_client_secret_github

GITHUB_CALLBACK_URL=http://localhost:3000/auth/github/callback

Pasos de configuración

Configuración de Google Cloud Console

1. Acceder a <https://console.cloud.google.com>
2. Crear nuevo proyecto: WhoAreYa-OAuth
3. Ir a APIs y Servicios > Credenciales
4. Crear ID de cliente OAuth 2.0
5. Tipo de aplicación: Aplicación web
6. Configurar URI autorizados

Configuración de GitHub OAuth

1. Acceder a <https://github.com/settings/developers>
2. Crear nueva OAuth App
3. Configurar

- Application name: Who Are Ya? - Dev
- Homepage URL: <http://localhost:3000>
- Authorization callback
URL: <http://localhost:3000/auth/github/callback>

Implementación

Modificaciones al modelo User

Se añadieron campos para manejar usuarios OAuth:

- isOAuthUser: Boolean para identificar usuarios OAuth
- oauthProvider: String con el proveedor (google/github)
- oauthId: String con el ID del usuario en el proveedor

Flujo de autenticación

1. Usuario hace clic en botón Google/GitHub
2. Redirigido al proveedor para autorización
3. Callback a nuestra aplicación con datos del usuario
4. Passport busca usuario por email:
 - Si existe: inicia sesión
 - Si no existe: crea nuevo usuario
5. Redirige según rol del usuario

Archivos creados/modificados

Archivos nuevos

- src/config/passport.js - Configuración de estrategias OAuth
- src/routes/oauthRoutes.js - Rutas para autenticación OAuth
- src/controllers/oauthController.js - Controlador para callbacks
- public/auth/css/oauth.css - Estilos para botones OAuth

Archivos modificados

- src/models/User.js - Añadidos campos para OAuth
- src/app.js - Inicialización de Passport
- views/auth/login.ejs - Añadidos botones OAuth

- package.json - Dependencias de Passport

El sistema de autenticación OAuth está completamente integrado con nuestro backend existente. Los usuarios pueden autenticarse con sus cuentas de Google o GitHub sin necesidad de registro manual, mientras mantenemos nuestro sistema de roles y sesiones existente.

2 Tests

¿Qué hemos hecho?

Configuración de Testing

Hemos implementado un sistema completo de pruebas para el backend utilizando las siguientes herramientas:

- Jest: Framework de testing para JavaScript
- Supertest: Para pruebas de integración de rutas HTTP
- MongoDB Memory Server: Base de datos en memoria para pruebas aisladas
- Cross-env: Para manejo de variables de entorno en diferentes sistemas

Estructura de Tests

tests/

```
├── setup.js          # Configuración de BD en memoria para tests
├── test-app.js       # Versión de la app sin Passport para tests
├── auth.test.js      # Tests del modelo User
├── player-model.test.js # Tests del modelo Player
├── auth-routes.test.js # Tests de rutas de autenticación
└── controllers.test.js # Tests de controladores públicos
```

Tests Implementados

1. Tests de Modelos (auth.test.js, player-model.test.js)
 - Modelo User:
 - Creación de usuario válido

- Búsqueda por email
- Validación de email único
- Validación de contraseña mínima (8 caracteres)
- Modelo Player:
 - Creación con datos mínimos
 - Creación con todos los campos
 - Validación de posición (solo DF, MF, FW, GK)
 - Campos opcionales pueden estar vacíos

2. Tests de Rutas de Autenticación (auth-routes.test.js)

- POST /register:
 - Registro exitoso de nuevo usuario
 - Fallo si email ya existe
 - Fallo si contraseñas no coinciden
- POST /login:
 - Login exitoso con credenciales correctas
 - Fallo con credenciales incorrectas

3. Tests de Controladores Públicos (controllers.test.js)

- GET /api/players: Listado de jugadores con paginación
- GET /api/players?search=: Filtrado por nombre (si está implementado)
- GET /api/players/:id: Obtención de jugador específico
- GET /api/game/current: Obtención del juego actual

Scripts de Testing

En package.json:

```
"scripts": {
  "test": "cross-env NODE_ENV=test jest --verbose",
  "test:watch": "cross-env NODE_ENV=test jest --watch",
  "test:coverage": "cross-env NODE_ENV=test jest --coverage"
```

```
}
```

Ejecución de Tests

Ejecutar todos los tests

```
npm test
```

Ejecutar tests específicos

```
npm test -- tests/auth.test.js
```

```
npm test -- tests/controllers.test.js
```

Configuración Técnica

Jest Configuration (jest.config.js)

```
module.exports = {  
  testEnvironment: 'node',  
  testMatch: ['**/tests/**/*.test.js'],  
  setupFilesAfterEnv: ['<rootDir>/tests/setup.js'],  
  verbose: true,  
  forceExit: true  
};
```

Cobertura de Tests

Los tests cubren:

- Validaciones de modelos
- Operaciones CRUD básicas
- Rutas de autenticación
- Rutas públicas del juego
- Manejo de errores HTTP
- Sesiones y autenticación

Notas Importantes

- Todas las pruebas son independientes y no dejan datos residuales
- La base de datos real del proyecto no es afectada por los tests

3. JSON Web Tokens

¿Qué hemos hecho?

En este milestone hemos implementado autenticación basada en JSON Web Tokens (JWT) para la API, permitiendo un sistema stateless donde el cliente incluye el token en cada petición protegida.

1. Generación de JWT al iniciar sesión

- Al hacer login (POST /login), el backend genera un JWT firmado con una clave secreta.
- El token incluye el userId, email y role.
- El cliente debe guardar el token (por ejemplo, en localStorage) y enviarlo en el header.
- Authorization en futuras peticiones.

2. Validación de JWT en rutas protegidas

- Se ha creado un middleware isAuthenticated que:
 - Extrae el token del header Authorization: Bearer.
 - Verifica la validez y firma del token usando la clave secreta.
 - Si es válido, añade los datos del usuario a req.user y permite el acceso.
 - Si no, responde con error 401.
- El middleware isAdmin comprueba que el usuario autenticado tenga rol admin.

3. Logout y expiración

- El logout en modo JWT es stateless: el cliente simplemente borra el token.
- Los tokens tienen expiración configurable (JWT_EXPIRES_IN).

4. Configuración necesaria

Añade estas variables al archivo .env:

JWT_SECRET

JWT_EXPIRES_IN

JWT_REFRESH_SECRET

JWT_REFRESH_EXPIRES_IN

5. Flujo de autenticación con JWT

1. El usuario hace login con email y contraseña.
2. El backend responde con un JWT si las credenciales son correctas.
3. El cliente guarda el token y lo envía en cada petición protegida.
4. El backend valida el token en cada request.

6. Archivos creados/modificados

Archivos nuevos:

- jwt.js
- Funciones para generar y verificar JWT.

Archivos modificados:

- authController.js
- Generación y validación de JWT en login, logout y obtención de usuario actual.
- authMiddleware.js
- Middleware para proteger rutas usando JWT.
- authRoutes.js - Soporte para login/logout con JWT.
- login.js y register.js
- Guardan el token JWT en el cliente tras login/registro.
- index.html
- Lógica para enviar el token en logout y refrescar el estado del usuario.
- package.json
- Añadida dependencia jsonwebtoken.

7. Ejemplo de uso

Login:

POST /login

Content-Type: application/json

```
{  
  "email": "usuario@ejemplo.com",  
  "password": "password123"  
}
```

Respuesta:

```
{  
  "success": true,  
  "data": {  
    "token": "<JWT_TOKEN>",  
    "user": {  
      "id": "...",  
      "name": "...",  
      "role": "user"  
    }  
  }  
}
```

Petición protegida:

GET /api/players

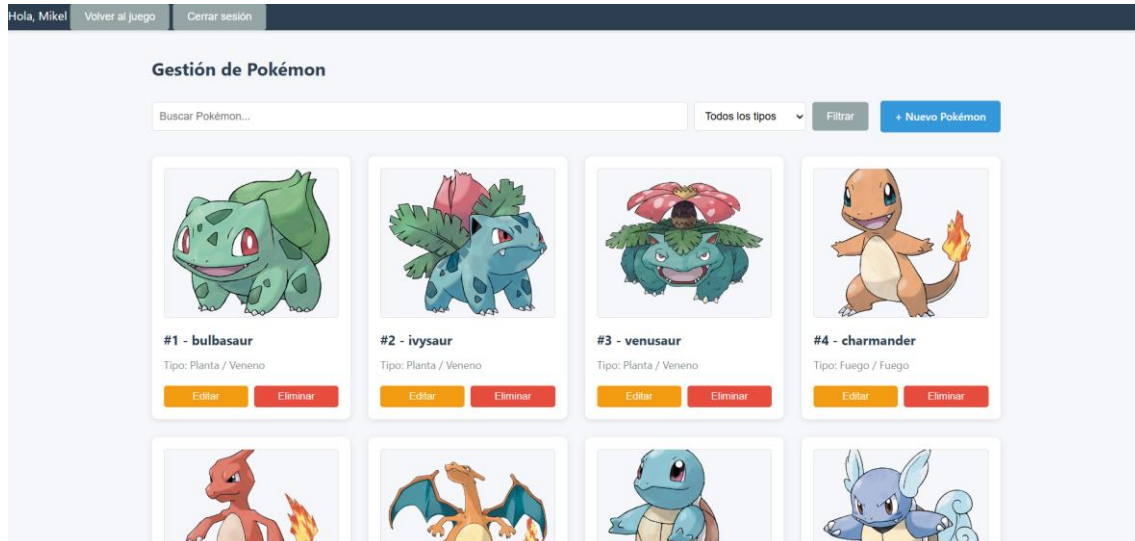
Authorization: Bearer <JWT_TOKEN>

El sistema JWT está completamente integrado y convive con el sistema de sesiones y OAuth, permitiendo autenticación flexible y segura para la API REST.

WholsThisPokemon?

- Tag de la milestone: https://github.com/RubenGGBC/WhoAreYaSW_Backend/releases/tag/backend-WholsThisPokemon

Ya



Este es un apartado **bonus** que el grupo a decidido hacer, ya que en la parte 1 del proyecto (frontend) se decidió realizar la **milestone 9**. Esta milestone consistía en reconstruir el proyecto WhoAreYa en un juego similar, pero con Pokémons en vez de con jugadores de fútbol. En esta segunda parte (backend) del proyecto, se ha vuelto a reconstruir todo el proyecto entero en un juego de Pokémon en vez de con jugadores de fútbol.

1. Localización y estructura:

- Todo este segundo juego de Pokémon se ha realizado dentro de la carpeta **WholsThisPokemon**, dentro del directorio raíz.
- La estructura es exactamente la misma que el juego WhoAreYa, es decir, la milestone 0 es exactamente la misma para el juego WhoAreYa que para WholsThisPokemon.

2. Metodología de trabajo y documentación:

El juego de Pokémon se ha realizado en paralelo al juego original (WhoAreYa) en una Branch aparte del repositorio. Se ha ido trabajando milestone por milestone,

como el juego original de WhoAreYa y se ha intentado replicar la misma lógica en todas ellas con respecto a las milestones del juego original.

Para poder encontrar más información acerca de lo que se ha realizado en cada una de las milestones, está disponible un readme dentro de la carpeta WhoAreYa (WhoAreYa/README.md).

3. Cómo probarlo:

- **Instalación y Configuración**

- **Requisitos previos:**

- Node.js

- MongoDB

- npm

- **Pasos de instalación:**

- 0. Cambiar a la carpeta WhoIsThisPokemon (partiendo del directorio raíz):

- `cd ../WhoIsThisPokemon/`

- 1. Instalar dependencias:

- `npm install`

- 2. Configurar variables de entorno:

- `cp .env.example .env`
`# Editar .env con tus valores`

- 3. Inicializar la base de datos:

- `npm run seed`

- 4. Iniciar el servidor:

- `node server.js`

El servidor estará disponible en <http://localhost:3001> (o en el que se haya configurado en las variables de entorno correspondientes).

****Notas**:**

- Para evitar problemas con la ejecución del proyecto, se recomienda poner un **puerto diferente** en ambos proyectos. Variables recomendadas (seguir el .env.example):
 - En .env raíz (WhoAreYa):
 - PORT=3000 y CORS_ORIGIN=http://localhost:3000
 - En .env de la carpeta WholsThisPokemon:
 - PORT=3001 y CORS_ORIGIN=http://localhost:3001
- Aunque los Pokémons y sus soluciones se guardan en una nueva dirección en la base de datos (si se sigue el .env.example: mongodb://localhost:27017/pokemon), los usuarios se seguirán guardando dentro del modelo de users del juego WhoAreYa (si se sigue el .env.example: mongodb://localhost:27017/whoareya).
- Para más información acerca de este juego:
https://github.com/RubenGGBC/WhoAreYaSW_Backend/blob/master/WholsThisPokemon/README.md

Uso de agentes

A pesar de que la mayor parte del proyecto lo hemos realizado los 3 integrantes del grupo, los agentes de IA nos han ayudado en distintos apartados como:

- CSS (Prácticamente todo ello)
- HTML (Gran ayuda)
- Feedback y mensajes de error (Ayuda parcial)
- Sistema de filtrado en la búsqueda de los jugadores (Ayuda parcial)
- Documentación en el readme (Sobre todo con el formato .md)
- Bugs puntuales

Conclusiones

Estamos **orgullosos del trabajo realizado** y consideramos que hemos realizado un buen trabajo como grupo:

- Se han realizado **todas y cada una de las milestones** disponibles o posibles, incluyendo todas las opcionales y el extra de WholsThisPokemon.
- Todo se ha realizado en el **plazo indicado** y ha sido entregado a tiempo.
- No nos hemos quedado con la sensación de que algo falla o está incompleto.
- Se ha trabajado de manera **ordenada y limpia**.
- Se ha aprendido aplicar todos los contenidos de la parte del backend de la asignatura en un proyecto real:
 - MongoDB
 - Mongoose
 - Formularios html
 - Ejs
 - Uso de api y peticiones HTTP
 - Uso de middlewares
 - JavaScript
 - Testing con Jest y Supertest
 - Express

Como futuras mejoras, quedaría añadir el único “pero” que hemos dejado en el proyecto:

- Se podría añadir las imágenes de las ligas a los selects del panel del admin, ya que sus opciones actualmente son Liga X (siendo X un número) y no se puede identificar qué liga es exactamente. Por eso quisimos añadir sus imágenes (tal y como hicimos con los equipos de fútbol). Sin embargo, el problema es que el id de la liga en fullplayers25.json y en leagues.txt (ambos archivos nos han sido dados en la asignatura y se nos pidió usar ambos) son diferentes, por lo que las imágenes de las ligas se guardan con un id y las ligas en MongoDB con otro. Esto nos lleva a que al acceder a las urls de las fotos guardadas en MongoDB dichas urls no lleven a ningún lado. Como consideramos que ante esto no podíamos hacer nada, lo hemos dejado sin fotos y con los nombres Liga X. Creemos que esto podría ser algo que se pueda cambiar de cara a los años siguientes (poner mismos ids en los dos archivos mencionados).